App Dev II Project Report

1. Student Details

Name: PRANAY KUMAR JENA

Roll Number: 24F3001877

Email: 24f3001877@ds.study.iitm.ac.in

About Me: I am a Standalone learner at IIT Madras BS Degree program with a deep interest in web application development and data-driven technologies. I enjoy building meaningful applications that combine learning, analytics, and user experience.

2. Project Details

Project Title: Placement Portal Application — V2 (RecruitRadar)

Problem Statement:

Institutes need efficient systems to manage campus recruitment activities involving companies, students, and placement drives. Manual coordination through spreadsheets and emails makes it difficult to manage company approvals, drive approvals, application tracking, interview scheduling, and placement records at scale.

Approach:

Building on the V1 submission, the app was rebuilt as RecruitRadar V2 using Flask (REST API backend) and Vue 3 (CDN, no build step) as the frontend, following a modular blueprint structure. It introduces role-based JWT authentication, a full drive-approval workflow, asynchronous background jobs using Celery and Redis, and Redis-based API caching , extending the V1 scope from a basic CRUD app into a production-style placement portal.

3. AI/LLM Declaration

I used Claude to assist in scaffolding SQLAlchemy models, structuring Celery task definitions, and debugging integration issues between Flask, Celery, and Redis across separate process boundaries.

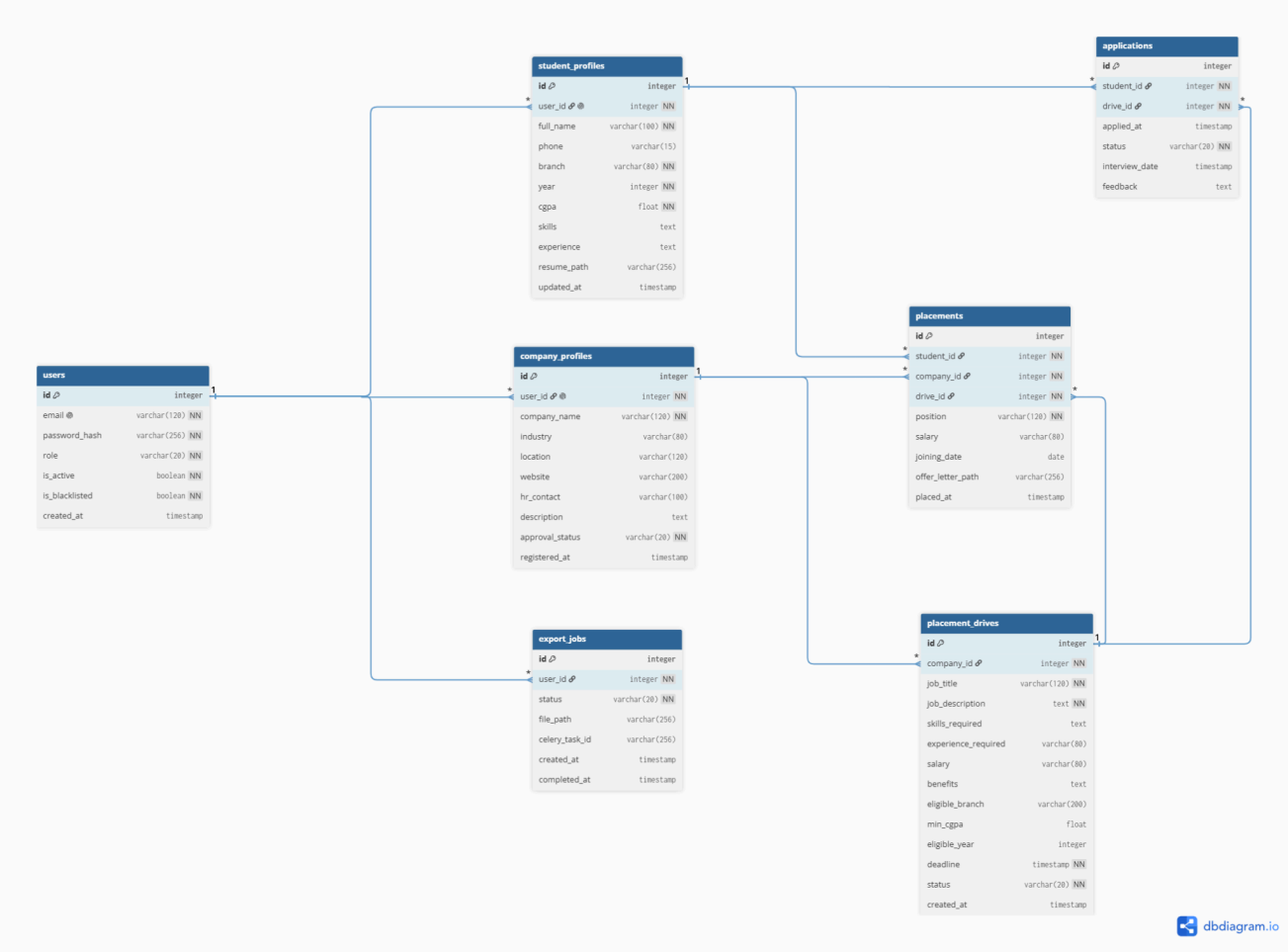
The extent of AI/LLM usage is around 15–20%, limited to code suggestions, debugging guidance, and documentation formatting.

All final implementation logic, testing, and integration were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend REST API framework
Flask-SQLAlchemy	Object Relational Mapper for SQLite database
Flask-JWT-Extended	Role-based authentication using JWT tokens
Vue 3 (CDN)	Reactive frontend UI, no build step required
Vue Router (Hash mode)	Client-side routing with role-based navigation guards
Bootstrap 5	Frontend styling and responsive design
SQLite	Lightweight local relational database
Redis	API response caching and Celery message broker
Celery + Celery Beat	Asynchronous and scheduled background jobs
Flask-Mail + Mailpit	Email delivery for reminders, reports, and alerts (local testing)
fpdf2 / pdfplumber	Offer letter PDF generation and resume text extraction (ATS checker)
Jinja2	Single entry-point HTML template only, not used for UI rendering

5. Database Schema / ER Diagram



Tables & Field Descriptions

- **users:** Central authentication table storing email, hashed password, and role (admin / company / student). Includes is_active and is_blacklisted flags for access control.
- **student_profiles:** Extends users for students - branch, year, cgpa, skills, experience, and resume_path.
- **company_profiles:** Extends users for companies - industry, location, hr_contact, and approval_status (pending / approved / rejected).
- **placement_drives:** Job postings created by companies - eligibility (branch, cgpa, year), deadline, and status (pending / approved / closed).
- **applications:** Junction table linking students to drives, with status lifecycle (applied → shortlisted → interview → placed / rejected) and a unique constraint on (student_id, drive_id) to block duplicate applications.
- **placements:** Created once an application is marked placed - stores salary, joining_date, and offer_letter_path.
- **export_jobs:** Tracks user-triggered async CSV export tasks - status, file_path, and celery_task_id.

Key Relationships

- One-to-One (1:1): Between users and student_profiles / company_profiles.
- One-to-Many (1:M): company_profiles → placement_drives, student_profiles → applications, placement_drives → applications, student_profiles → placements.

6. API Resource Endpoints

Endpoint	Method	Description
`/api/auth/register`	**POST**	Registers a Student or Company account with role-specific validation.
`/api/auth/login`	**POST**	Authenticates credentials and issues a JWT access token with role claims.
`/api/admin/companies/<id>/approve`	**PUT**	Approves a pending company registration.
`/api/admin/drives/<id>/approve`	**PUT**	Approves a pending placement drive, making it visible to students.
`/api/company/drives`	**POST**	Creates a new placement drive with status set to 'pending'.
`/api/student/drives/<id>/apply`	**POST**	Validates CGPA/branch/year eligibility and deadline before creating an application.
`/api/company/applications/<id>`	**PUT**	Updates application status, interview date, and feedback.
`/api/student/export`	**POST**	Triggers an async Celery job to export application history as CSV.
`/api/student/ats-check`	**POST**	Extracts resume text via pdfplumber and matches it against a drive's required skills.
`/api/student/profile`	**GET/PUT**	Retrieves or updates the student profile, including resume upload.

7. Architecture and Features

Architecture Overview:

- **app.py:** Flask application factory at the project root, registering all blueprints and initializing extensions.
- **Modular blueprints:** Separate route files (auth, admin, company, student) instead of a monolithic file.
- **extensions.py:** Centralised initialisation of db, jwt, mail, cors, and redis_client to avoid circular imports.
- **tasks.py / celery_worker.py:** Celery task definitions and worker entry point, run as a separate process from the Flask app.
- **frontend/src:** Vue 3 components and page files loaded via CDN script tags, mounted into a single Jinja2 entry point.

Implemented Features:

- Automated Environment Setup: init_db.py creates all tables programmatically and seeds the single Admin account.
- Multi-Stage Approval Workflow: Companies and placement drives require explicit Admin approval before becoming visible, now managed directly from the Admin Dashboard.
- Student Eligibility Guard: Automatic verification of CGPA, branch, and year against drive requirements during application, with duplicate-application prevention at the database level.
- Reputation Management: Blacklist/deactivate functionality for both students and companies.
- Background Jobs : Daily deadline reminders and monthly HTML placement reports via Celery Beat, plus user-triggered async CSV export with email alert on completion.
- Redis Caching : Cached dashboard statistics and search endpoints with TTL-based expiry, degrading gracefully when Redis is unavailable.
- ATS Resume Checker : Keyword-matching resume screener comparing a student's resume against a drive's required skills, with a transparent match score.
- Offer Letter Generation : Dummy PDF offer letters generated for placed students using fpdf2.

8. Video Presentation

Drive Link: Click Here

AppDev2_PPAV2_Demo.mp4